

EXPERIMENT-4

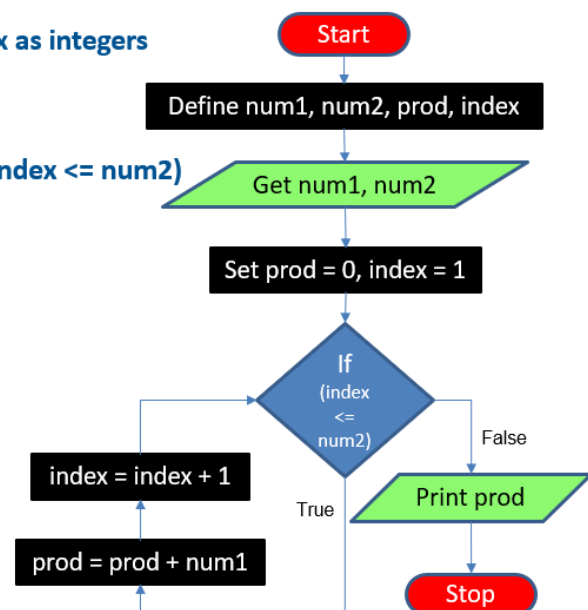
IN this experiment we are going to find a mistake in a Algorithm and a flowchart then we will fix the mistake and add the values given to check and run the algorithm



Multiply 2 numbers without using *operator (9)

• Algorithm & Flowchart

1. Define num1, num2, prod, index as integers
2. Get num1, num2
3. Set prod = 0, index = 1
4. Repeat steps 4.1 and 4.2 until (index <= num2)
 - 4.1. prod = prod + num1
 - 4.2. index = index + 1
5. Print prod
6. Stop



First we will be fixing this algorithm

1. Define num1, num2, prod, index as integers
2. Get num1, num2
3. If $|num1| < |num2|$ (assigning smaller value to num2)
 - 3.1 YES- temp1 = num1, num1 = num2, num2 = temp1

3.2 NO- $\text{num1} = \text{num1}$, $\text{num2} = \text{num2}$

4. Set $\text{prod} = 0$, $\text{index} = 1$

5. repeat steps 5.1 and 5.2 until $\text{index} \leq \text{num2}$

5.1. $\text{Prod} = \text{prod} + \text{num1}$

5.2. $\text{index} = \text{index} + 1$

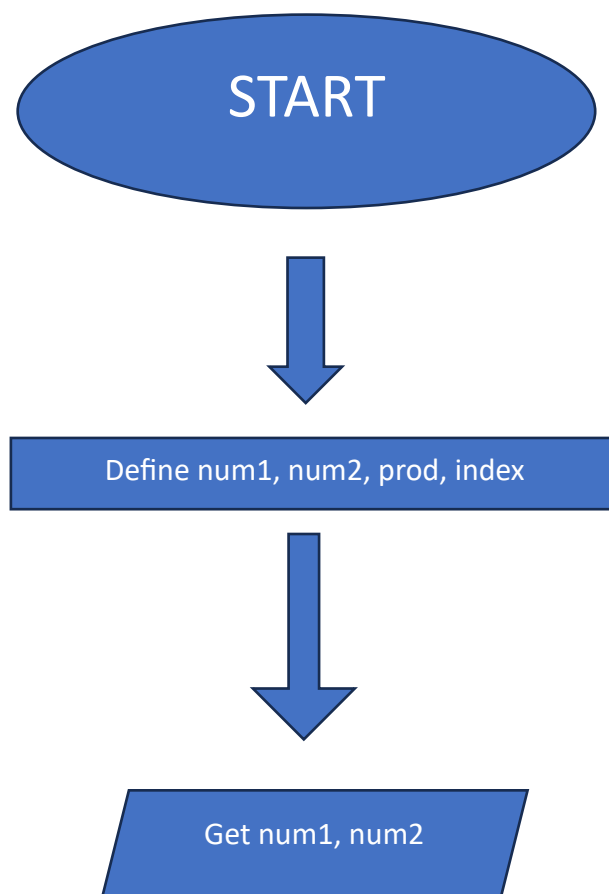
6. If exactly one number is negative then

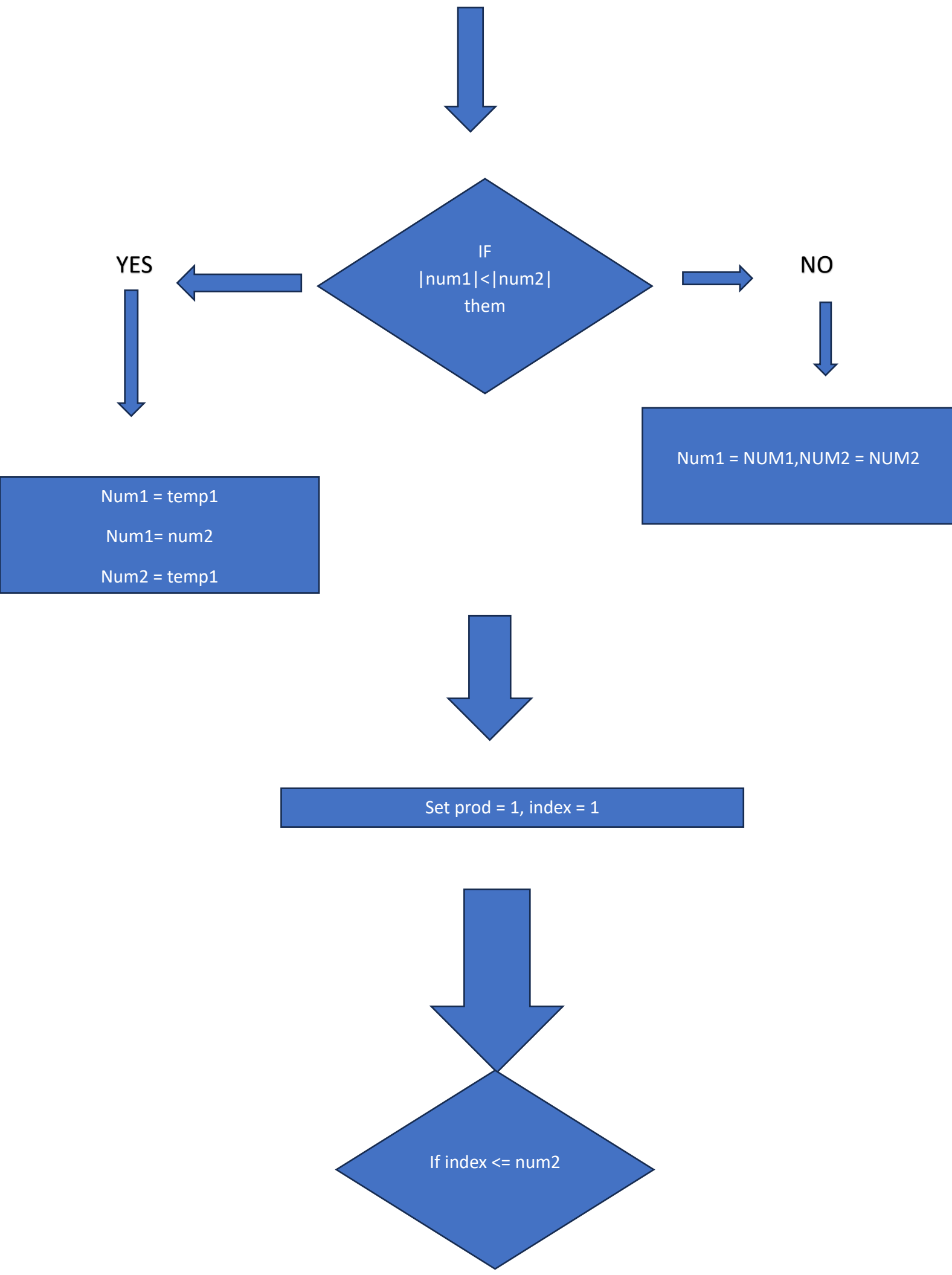
6.1. $\text{prod} = -\text{prod}$

7. print prod

8. stop

Now that the algorithm has been fixed we will fix the flowchart according to this pseudocode

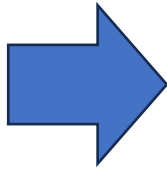




True

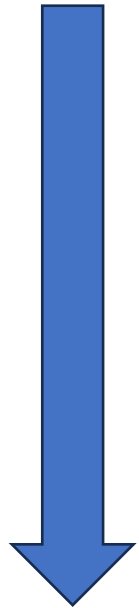


Prod = prod + 1

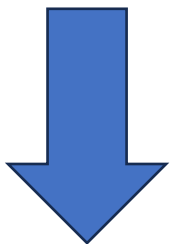


INDEX = index + 1

False



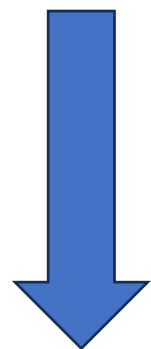
TRUE



Prod = - prod

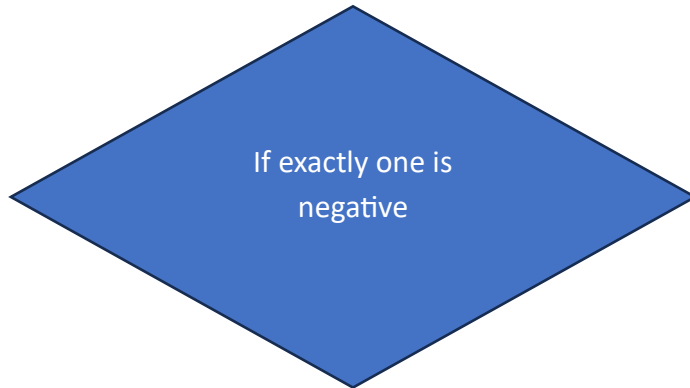


False



Print prod

If exactly one is
negative



STOP



Now we will try this code on some of the cases to verify

Case-1 input = 12,3 and expected product=36

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int num1 = 12;
6     int num2 = 3;
7     int prod, index, temp;
8
9     if ((num1 < 0 ? -num1 : num1) < (num2 < 0 ? -num2 : num2))
10    {
11        temp = num1;
12        num1 = num2;
13        num2 = temp;
14    }
15
16    prod = 0;
17    index = 1;
18
19    while (index ≤ (num2 < 0 ? -num2 : num2))
20    {
21        prod = prod + (num1 < 0 ? -num1 : num1);
22        index++;
23    }
24
25    if ((num1 < 0 && num2 ≥ 0) || (num1 ≥ 0 && num2 < 0))
26    {
27        prod = -prod;
28    }
29
30    printf("Input: %d %d\n", num1, num2);
31    printf("Product = %d\n", prod);
32
33    return 0;
34 }
```

```
Session Actions Edit View Help
(kali㉿kali)-[~]
$ gedit case1.c
** (gedit:9162): WARNING **: 14:18:20.400: Could not load Peas
** (gedit:9162): WARNING **: 14:18:20.400: Could not load Peas
(kali㉿kali)-[~]
$ cc -g case1.c
(kali㉿kali)-[~]
$ ./a.out
Input: 12 3
Product = 36
```

CASE-2 INPUT = -9,-4

```

1 #include <stdio.h>
2
3 int main()
4 {
5     int num1 = -9;
6     int num2 = -4;
7     int prod, index, temp;
8
9     if ((num1 < 0 ? -num1 : num1) < (num2 < 0 ? -num2 : num2))
10    {
11        temp = num1;
12        num1 = num2;
13        num2 = temp;
14    }
15
16    prod = 0;
17    index = 1;
18
19    while (index ≤ (num2 < 0 ? -num2 : num2))
20    {
21        prod = prod + (num1 < 0 ? -num1 : num1);
22        index++;
23    }
24
25    if ((num1 < 0 && num2 ≥ 0) || (num1 ≥ 0 && num2 < 0))
26    {
27        prod = -prod;
28    }
29
30    printf("Input: %d %d\n", num1, num2);
31    printf("Product = %d\n", prod);
32
33    return 0;
34 }

```

```

(kali㉿kali)-[~]
$ gedit case2.c

```

```

** (gedit:14878): WARNING **: 14:23:28.124: Could not load
** (gedit:14878): WARNING **: 14:23:28.124: Could not load

```

```

(kali㉿kali)-[~]
$ cc -g case2.c

```

```

(kali㉿kali)-[~]
$ ./a.out
Input: -9 -4
Product = 36

```

```

(kali㉿kali)-[~]
$

```